

Punto de Venta en Python
Erick Rolando Dominguez Marquez
326820



Contenido

1.	Introducción.....	1
2.	Objetivos del proyecto.....	1
3.	Conceptos Teóricos.....	1
4.	Código.....	1
5.	Funcionalidades.....	14
6.	Instrucciones de Ejecución.....	17
7.	Conclusiones.....	17

Introducción

El documento describe las funciones y los requisitos del sistema de punto de venta así como una descripción detallada de su ejecución y que conceptos fueron aplicados.

Objetivos del proyecto

El objetivo principal es desarrollar un sistema de punto de venta capaz de registrar y gestionar las ventas de productos, controlar el stock y alertar automáticamente cuando un producto tenga bajo inventario. Además, el sistema debe respaldar las ventas y los tickets generados en archivos de texto con su respectiva fecha, así como mantener actualizado el stock también en un archivo de respaldo.

Conceptos Teóricos

- Arreglos
 - Se utilizaron arreglos para almacenar los registros de los tickets y los productos por su facilidad y por no tener un caso de uso más complejo que requiere estructuras como un diccionario

```
productos = []
historial_ventas = []
#DICCIONARIO - para llevar un registro de los productos

historial_ventas.append(venta.strip())
venta = ""

stock = int(partes[2].replace("stock: ", ""))
productos.append({'nombre': nombre, 'precio': precio, 'stock': stock})
ept :
```

- Ciclo for y while
 - Se utilizaron ciclos while tanto para guardar y leer los registros de los productos y ventas como para mantener activos a los hilos utilizados

```
while True:
    with open("productos.txt", "w") as f:
        for prod in productos:
            f.write(f"{prod['nombre']} - ${prod['precio']} - Stock: {prod['stock']}\n")

    with open("ventas.txt", "w") as f:
        for venta in historial_ventas:
            f.write(venta + "\n")

# HILO 2 - STOCK
def hilo_stock_critico():
    #Uso de WHILE True para mantener los hilos con vida por su uso de daemon
    while True:
        for prod in productos:
            nombre = prod['nombre']
            if prod['stock'] <= 2 and not productos_alertados.get(nombre, False):
                productos_alertados[nombre] = True
```

- También se utilizaron ciclos for para operaciones con arreglos como buscar elementos que se manipulan o para listarlos como es el caso del carrito

```

for i, cant in carrito:
    prod = productos[i]
    subtotal = prod['precio'] * cant
    ticket += f"{prod['nombre']} x{cant} = ${subtotal:.2f}\n"
    total += subtotal
    prod['stock'] -= cant
    if prod['stock'] < 2:
        productos_alertados[prod['nombre']] = False
ticket += f"\nTOTAL: ${total:.2f}"
ticket += f"\n ----- \n"

```

```

nombre = seleccion.get()
for prod in productos:
    if prod['nombre'] == nombre:
        try:
            cantidad = int(entry_aumento.get())
            if cantidad <= 0:
                raise ValueError
        except ValueError:
            messagebox.showerror("Error", "Ingresa una cantidad valida")
            return

        prod['stock'] += cantidad
        productos_alertados[nombre] = False
        messagebox.showinfo("Exito", f"Nuevo stock de '{nombre}': {prod['stock']}")
        ventana.destroy()
        break

```

- Condicionales if

- Se utilizaron condicionales if para evitar errores o para el manejo de casos específicos como lo es el carrito, en caso de que el arreglo este vacío se nos muestra una advertencia, o el stock en donde si el stock del producto actual es menor a 2 procederá a modificar el diccionario de productos_alertados para que uno de nuestros hilos nos pueda alertar que el stock está bajo

```

if not carrito:
    messagebox.showwarning("Aviso", "El carrito esta vacio")
    return

```

```

prod['stock'] -= cant
if prod['stock'] < 2:
    productos_alertados[prod['nombre']] = False

```

- Diccionarios

- Se utilizó un diccionario para mantener el control de qué productos ya han sido alertados de tener un stock bajo, de esta manera podemos buscar por el nombre del producto y saber si se encuentra de una manera más eficaz que si se usara un arreglo

```
#Diccionario para alertar
productos_alertados = {}

if prod['stock'] <= 2 and not productos_alertados.get(nombre, False):
    productos_alertados[nombre] = True
```

- Excepciones

- Han sido utilizadas bloques de try y catch en la mayoría de los procesos del código como los inputs que maneja el usuario o en caso de fallar una manipulación de datos, uno de los bloques más interesantes es el try/catch de la función cargar_registros() donde la excepción se maneja con un continue, esto se debe a que en caso de tener datos corruptos se continúe leyendo la siguiente línea de texto

```
nombre = entry_nombre.get()
try:
    precio = float(entry_precio.get())
    stock = int(entry_stock.get())
except ValueError:
    messagebox.showerror("Error", "Precio y stock deben ser numericos")
    return

try:
    partes = linea.strip().split(" - ")
    nombre = partes[0]
    precio = float(partes[1].replace("$", ""))
    stock = int(partes[2].replace("Stock: ", ""))
    productos.append({'nombre': nombre, 'precio': precio, 'stock': stock})
except:
    continue
```

- Hilos

- En este programa contamos con 3 hilos aparte del hilo principal, el primer hilo es responsable de mantener el registro de la hora, el segundo hilo es responsable de estar supervisando el stock y de alertarnos en caso de que uno de los productos tenga un stock menor. El tercer hilo es responsable de mantener un registro de las ventas y los productos cada cierta cantidad de tiempo para

asegurarnos de no tener ninguna perdida de información.

```
# HILO 1 - RELOJ
def hilo_reloj():
    #Uso de While True para mantener los hilos con vida por su uso de daemon
    while True:
        hora_actual = time.strftime("%H:%M:%S")
        label_reloj.config(text=f"Hora: {hora_actual}")
        time.sleep(1)

# HILO 2 - STOCK
def hilo_stock_critico():
    #Uso de WHILE True para mantener los hilos con vida por su uso de daemon
    while True:
        for prod in productos:
            nombre = prod['nombre']
            if prod['stock'] <= 2 and not productos_alertados.get(nombre, False):
                productos_alertados[nombre] = True
                ventana = tk.Toplevel(root)
                ventana.title("ALERTA")
                tk.Label(ventana, text=f"ALERTA STOCK BAJO").pack(pady=10)
                tk.Label(ventana, text=f"El producto '{prod['nombre']}' tiene bajo stock: {prod['stock']}").pack(pady=10)
                tk.Button(ventana, text="Añadir stock?", command=anadir_stock).pack(pady=5)
            time.sleep(10)

# HILO 3 - RESPALDO AUTOMATICO
def hilo_guardar_registros():
    #Uso de WHILE True para mantener los hilos con vida por su uso de daemon
    while True:
        with open("productos.txt", "w") as f:
            for prod in productos:
                f.write(f"{prod['nombre']} - ${prod['precio']} - Stock: {prod['stock']}\n")

        with open("ventas.txt", "w") as f:
            for venta in historial_ventas:
                f.write(venta + "\n")

        time.sleep(30)
```

Codigo

```
import tkinter as tk
from tkinter import messagebox
import threading
import time
import copy
import os

productos = []
historial_ventas = []

#DICCIONARIO - para llevar un control de que productos han sido alertados de su bajo stock
productos_alertados = {}

def cargar_registros():
    global productos, historial_ventas
    productos_alertados = {}
```

```

productos = []
historial_ventas = []

#CARGAR RESPALDO DE LOS PRODUCTOS
if os.path.exists("productos.txt"):
    with open("productos.txt", "r") as f:
        for linea in f:
            try:
                partes = linea.strip().split(" - ")
                nombre = partes[0]
                precio = float(partes[1].replace("$", ""))
                stock = int(partes[2].replace("Stock: ", ""))
                productos.append({'nombre': nombre, 'precio': precio, 'stock':
stock})
            except :
                continue

#CARGAR LAS VENTAS
if os.path.exists("ventas.txt"):
    with open("ventas.txt", "r") as f:
        venta = ""
        for linea in f:
            venta += linea
            if "-----" in linea:
                historial_ventas.append(venta.strip())
                venta = ""

# HILO 1 - RELOJ
def hilo_reloj():
    #Uso de While True para mantener los hilos con vida por su uso de daemon
    while True:
        hora_actual = time.strftime("%H:%M:%S")
        label_reloj.config(text=f"Hora: {hora_actual}")
        time.sleep(1)

# HILO 2 - STOCK
def hilo_stock_critico():
    #Uso de WHILE True para mantener los hilos con vida por su uso de daemon
    while True:
        for prod in productos:
            nombre = prod['nombre']

```

```

        if prod['stock'] <= 2 and not productos_alertados.get(nombre, False):
            productos_alertados[nombre] = True
            ventana = tk.Toplevel(root)
            ventana.title("ALERTA")
            tk.Label(ventana, text=f"ALERTA STOCK BAJO").pack(pady=10)
            tk.Label(ventana, text=f"El producto '{prod['nombre']}' tiene bajo
stock: {prod['stock']}").pack(pady=10)
            tk.Button(ventana, text="Añadir stock?",
command=anadir_stock).pack(pady=5)
            time.sleep(10)

# HILO 3 - RESPALDO AUTOMATICO
def hilo_guardar_registros():
    #Uso de WHILE True para mantener los hilos con vida por su uso de daemon
    while True:
        with open("productos.txt", "w") as f:
            for prod in productos:
                f.write(f"{prod['nombre']} - ${prod['precio']} - Stock:
{prod['stock']}\n")

            with open("ventas.txt", "w") as f:
                for venta in historial_ventas:
                    f.write(venta + "\n")

            time.sleep(30)

def registrar_producto():
    nombre = entry_nombre.get()
    try:
        precio = float(entry_precio.get())
        stock = int(entry_stock.get())
    except ValueError:
        messagebox.showerror("Error", "Precio y stock deben ser numericos")
    return

    productos.append({"nombre": nombre, "precio": precio, "stock": stock})
    productos_alertados[nombre] = False
    messagebox.showinfo("EXITO", f"Producto '{nombre}' registrado")
    entry_nombre.delete(0, tk.END)
    entry_precio.delete(0, tk.END)
    entry_stock.delete(0, tk.END)

```

```

def ventana_registro():
    ventana = tk.Toplevel(root)
    ventana.title("Registrar producto")

    tk.Label(ventana, text="Nombre del producto:").grid(row=0, column=0)
    tk.Label(ventana, text="Precio:").grid(row=1, column=0)
    tk.Label(ventana, text="Stock:").grid(row=2, column=0)

    global entry_nombre, entry_precio, entry_stock
    entry_nombre = tk.Entry(ventana)
    entry_precio = tk.Entry(ventana)
    entry_stock = tk.Entry(ventana)

    entry_nombre.grid(row=0, column=1)
    entry_precio.grid(row=1, column=1)
    entry_stock.grid(row=2, column=1)

    tk.Button(ventana, text="Guardar producto", command=registrar_producto).grid(row=3,
columnspan=2, pady=10)

def ver_catalogo():
    ventana = tk.Toplevel(root)
    ventana.title("Catalogo de Productos")

    if not productos:
        tk.Label(ventana, text="No hay productos registrados.").pack(pady=10)
        return

    tk.Label(ventana, text="Catalogo de productos").pack(pady=5)
    for index, prod in enumerate(productos, start=1):
        texto = f"{index}. {prod['nombre']} - ${prod['precio']:.2f} - Stock:
{prod['stock']}"
        tk.Label(ventana, text=texto).pack(anchor='w')

def realizar_venta():
    ventana = tk.Toplevel(root)
    ventana.title("Realizar Venta")

    carrito = []
    total_var = tk.StringVar()
    total_var.set("Total: $0.00")

```



```

if not productos:
    tk.Label(ventana, text="No hay productos.").pack(pady=10)
    return

productos_temporales = copy.deepcopy(productos)

def actualizar_etiquetas():
    for index, label in enumerate(etiquetas_stock):
        label.config(text=f"{productos_temporales[index]['nombre']} -
${productos_temporales[index]['precio']} - Stock:
{productos_temporales[index]['stock']}")

def agregar_a_carrito(index):
    try:
        cantidad = int(entry_cantidades[index].get())
        if cantidad <= 0 or cantidad > productos_temporales[index]['stock']:
            raise ValueError
    except ValueError:
        messagebox.showerror("Error", f"Cantidad invalida para
{productos_temporales[index]['nombre']}")
        return

    carrito.append((index, cantidad))
    productos_temporales[index]['stock'] -= cantidad
    carrito_listbox.insert(tk.END, f"{productos_temporales[index]['nombre']}
x{cantidad}")
    entry_cantidades[index].delete(0, tk.END)
    actualizar_etiquetas()
    calcular_total()

def eliminar_del_carrito():
    seleccion = carrito_listbox.curselection()
    if not seleccion:
        messagebox.showwarning("Aviso", "Selecciona un producto del carrito para
eliminar.")
        return

    index = seleccion[0]
    index, cantidad = carrito.pop(index)
    productos_temporales[index]['stock'] += cantidad
    carrito_listbox.delete(index)

```

```

    actualizar_etiquetas()
    calcular_total()

def calcular_total():
    total = sum(productos_temporales[i]['precio'] * c for i, c in carrito)
    total_var.set(f"Total: ${total:.2f}")

def confirmar_venta():
    if not carrito:
        messagebox.showwarning("Aviso", "El carrito esta vacio")
        return

    ticket = "Ticket de venta:\n"
    total = 0
    for i, cant in carrito:
        prod = productos[i]
        subtotal = prod['precio'] * cant
        ticket += f"{prod['nombre']} x{cant} = ${subtotal:.2f}\n"
        total += subtotal
        prod['stock'] -= cant
        if prod['stock'] < 2:
            productos_alertados[prod['nombre']] = False
    ticket += f"\nTOTAL: ${total:.2f}"
    ticket += f"\n ----- \n"

    historial_ventas.append(ticket)
    os.makedirs("tickets", exist_ok=True)

    filename = f"tickets/ticket_{time.strftime("%H:%M:%S")}.txt"
    with open(filename, "w") as f:
        f.write(ticket)

    messagebox.showinfo("Venta realizada", ticket)
    ventana.destroy()

tk.Label(ventana, text="Selecciona productos y cantidad:").pack(pady=5)
entry_cantidades = []
etiquetas_stock = []

for index, prod in enumerate(productos_temporales):
    frame = tk.Frame(ventana)
    frame.pack(anchor='w')

```

```

        etiqueta = tk.Label(frame, text=f"{prod['nombre']} - ${prod['precio']} - Stock:
{prod['stock']}")
        etiqueta.pack(side='left')
        etiquetas_stock.append(etiqueta)

        entry = tk.Entry(frame, width=5)
        entry.pack(side='left', padx=5)
        entry_cantidades.append(entry)

        tk.Button(frame, text="Agregar", command=lambda i=index:
agregar_a_carrito(i)).pack(side='left')

    # Lista del carrito
    tk.Label(ventana, text="Carrito:").pack(pady=5)
    carrito_listbox = tk.Listbox(ventana, width=40)
    carrito_listbox.pack()

    tk.Button(ventana, text="Eliminar del carrito",
command=eliminar_del_carrito).pack(pady=5)
    tk.Label(ventana, textvariable=total_var, font=("Helvetica", 12)).pack(pady=10)
    tk.Button(ventana, text="Confirmar venta", command=confirmar_venta).pack()

def ver_historial():
    ventana = tk.Toplevel(root)
    ventana.title("Historial de Ventas")

    if not historial_ventas:
        tk.Label(ventana, text="No hay ventas registradas.").pack(pady=10)
        return

    tk.Label(ventana, text="Historial de ventas", font=("Helvetica", 12)).pack(pady=5)

    for venta in historial_ventas:
        tk.Label(ventana, text=venta, justify='left', anchor='w').pack(padx=10,
anchor='w')

def anadir_stock():
    def actualizar_stock_actual(*args):
        nombre = seleccion.get()
        for prod in productos:

```

```

        if prod['nombre'] == nombre:
            entry_stock_actual.config(state='normal')
            entry_stock_actual.delete(0, tk.END)
            entry_stock_actual.insert(0, str(prod['stock']))
            entry_stock_actual.config(state='readonly')
            btn_confirmar.config(state='normal')
            break

def confirmar_aumento():
    nombre = seleccion.get()
    for prod in productos:
        if prod['nombre'] == nombre:
            try:
                cantidad = int(entry_aumento.get())
                if cantidad <= 0:
                    raise ValueError
            except ValueError:
                messagebox.showerror("Error", "Ingresa una cantidad valida")
                return

            prod['stock'] += cantidad
            productos_alertados[nombre] = False
            messagebox.showinfo("Exito", f"Nuevo stock de '{nombre}':
{prod['stock']}")
            ventana.destroy()
            break

    if not productos:
        messagebox.showinfo("Aviso", "No hay productos registrados.")
        return

    ventana = tk.Toplevel(root)
    ventana.title("Añadir stock")

    tk.Label(ventana, text="Selecciona producto:").grid(row=0, column=0, padx=5,
pady=5)

    nombres_productos = [p['nombre'] for p in productos]
    seleccion = tk.StringVar(ventana)
    seleccion.set(nombres_productos[0]) #SELECCION DE EL PRIMER PRODUCTO PARA ENSEÑAR
EN EL DROPDOWN
    menu = tk.OptionMenu(ventana, seleccion, *nombres_productos,
command=actualizar_stock_actual)

```

```

menu.grid(row=0, column=1, padx=5, pady=5)

tk.Label(ventana, text="Stock actual:").grid(row=1, column=0)
entry_stock_actual = tk.Entry(ventana, state='readonly')
entry_stock_actual.grid(row=1, column=1, padx=5, pady=5)

tk.Label(ventana, text="Cantidad a añadir:").grid(row=2, column=0)
entry_aumento = tk.Entry(ventana)
entry_aumento.grid(row=2, column=1, padx=5, pady=5)

btn_confirmar = tk.Button(ventana, text="Confirmar", command=confirmar_aumento)
btn_confirmar.grid(row=3, columnspan=2, pady=10)

actualizar_stock_actual() #STOCK DEL PRODUCTO SELECCIONADO

# VENTANA PRINCIPAL
root = tk.Tk()
root.title("Sistema de Punto de Venta")
root.geometry("320x300")

# MENU PRINCIPAL
tk.Label(root, text="SISTEMA DE PUNTO DE VENTA", font=("Helvetica", 14)).pack(pady=10)
label_reloj = tk.Label(root, text="", font=("Helvetica", 10))
label_reloj.pack()
tk.Button(root, text="1. Registrar producto", width=30,
command=ventana_registro).pack(pady=5)
tk.Button(root, text="2. Ver catalogo", width=30, command=ver_catalogo).pack(pady=5)
tk.Button(root, text="3. Añadir stock", width=30, command=anadir_stock).pack(pady=5)
tk.Button(root, text="4. Realizar una venta", width=30,
command=realizar_venta).pack(pady=5)
tk.Button(root, text="5. Ver historial de ventas", width=30,
command=ver_historial).pack(pady=5)
tk.Button(root, text="Salir", width=30, command=root.quit).pack(pady=10)

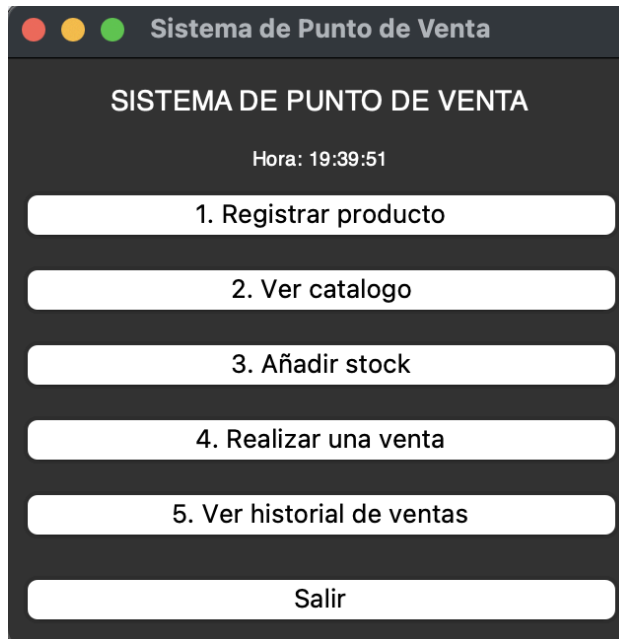
#AQUI CARGAMOS NUESTROS DATOS
cargar_registros()

#IINICIO HILOS
threading.Thread(target=hilo_reloj, daemon=True).start()
threading.Thread(target=hilo_stock_critico, daemon=True).start()
threading.Thread(target=hilo_guardar_registros, daemon=True).start()

```

```
#INICIO DE VENTANA TKINTER  
root.mainloop()
```

Funcionalidades



Al entrar al programa la primera pantalla que vemos es el menú principal donde podemos visualizar la hora y escoger una de las opciones de las operaciones a realizar. Si es nuestra primera vez usando el programa lo primero que tendríamos que hacer es registrar nuestros productos en la opción 1.

A screenshot of a Tkinter window titled 'Registrar producto'. The window has a dark gray background. It contains three input fields: 'Nombre del producto:' with the value 'Banana', 'Precio:' with the value '10', and 'Stock:' with the value '100'. Below these fields is a white button labeled 'Guardar producto'.

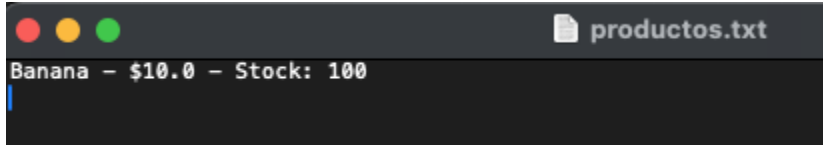
Hemos registrado con éxito nuestro primer producto, en caso de que los valores ingresen con un valor no aceptado se mostrará una alerta como la siguiente:



Una vez que se tenga registro de un producto en el stock es posible visualizarlo en el catálogo o incluso en nuestro archivo de respaldo.



Después de tener un producto en nuestro inventario el hilo responsable de hacer los respaldos actualizará el archivo de productos dentro de la misma carpeta. Este hilo corre concurrentemente cada 30 segundos verificando que el archivo esté actualizado



El programa también nos ayuda a llevar un control de la mercancía nueva que llega a la tienda así que tiene una función para ayudarnos a añadir stock a productos ya existentes. Solo seleccionamos un producto y nos informará con cuánto stock se cuenta en el momento y cuanto se añadirá



Añadir stock

Selecciona producto:

Banana

Stock actual:

100

Cantidad a añadir:

10

Confirmar

Nuevo stock de 'Banana': 110

OK

Si el stock a añadir es correcto tendremos una confirmación por parte del sistema. En caso de ingresar caracteres incorrectos se mostrará una alerta para informar al usuario

Ingresar una cantidad valida

OK

Cuando se necesite hacer una venta se nos mostrará un menú con el stock y los precios, solo ingresamos

4. Realizar una venta

Realizar Venta

Selecciona productos y cantidad:

Banana - \$10.0 - Stock: 110

Agregar

Manzana - \$10.0 - Stock: 70

Agregar

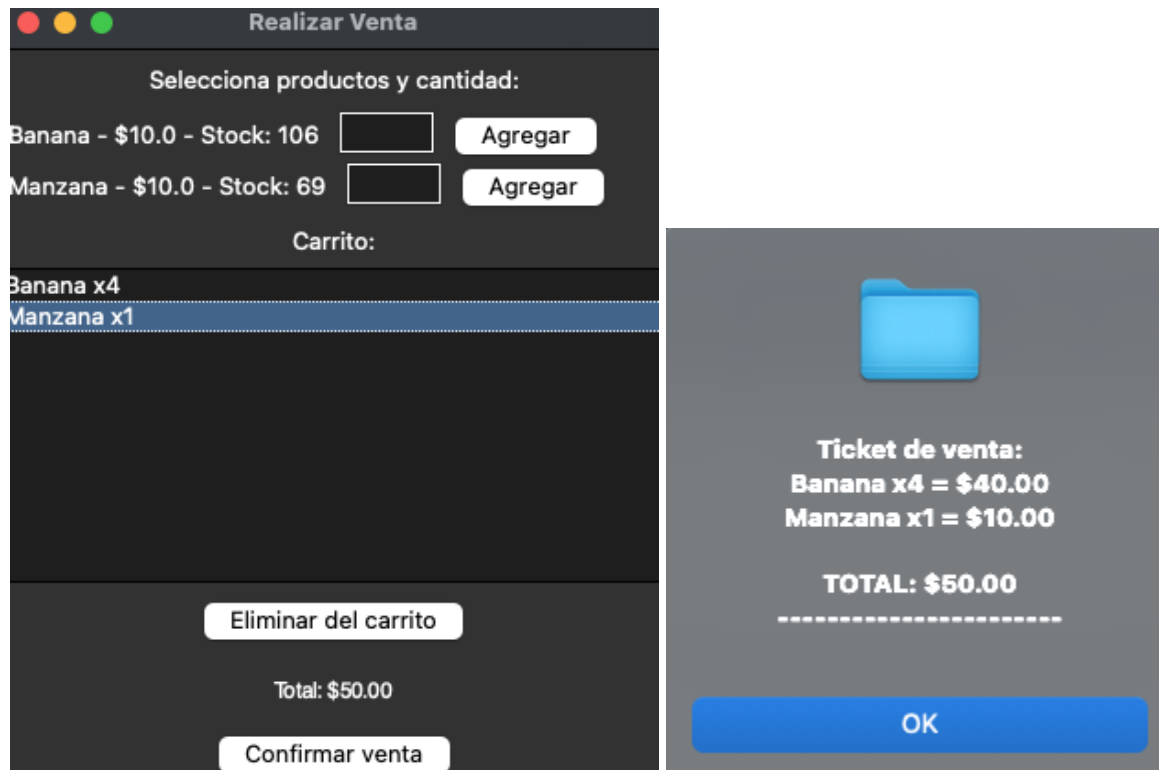
Carrito:

Eliminar del carrito

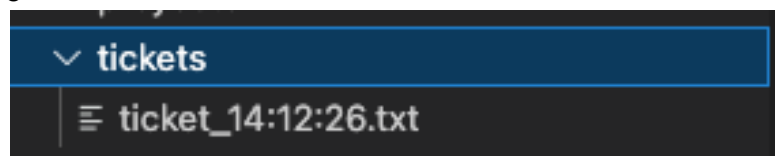
Total: \$0.00

Confirmar venta

Dentro de este menú se nos mostrará el listado de productos que agreguemos y se nos mostrará el total, dentro de este mismo menú podemos seguir agregando productos o eliminando del listado hasta poder confirmar la venta, al confirmar una venta se generará nuestro ticket.



Este ticket también podrá ser encontrado en la carpeta de tickets con la fecha en la que se generó.



Asi mismo si nuestra venta hizo que nuestro stock quedará con 2 o menos productos uno de nuestros hilos detectara esa cantidad y nos alertará que debemos pedir mas de el producto actual



Los registros generados se podrán encontrar en el historial de ventas, donde se mostrarán todas las ventas que han sido realizadas. Este mismo registro debe ser respaldado por uno de nuestros hilos y se debería encontrar en el archivo de ventas dentro de la misma carpeta que nuestro programa.



Instrucciones de ejecución

El código fue ejecutado y probado con python 3.12 y aunque los paquetes utilizados son paquetes estándares de python en caso de no contar con ellos se pueden instalar. Los paquetes requeridos son los siguientes:

```
tkinter  
threading  
time  
copy  
os
```

Una vez que tengamos nuestro ambiente configurado y nuestro código en un archivo .py solo abrimos nuestra consola y corremos el siguiente comando y tendremos

```
python nombre_del_archivo.py
```

En caso de contar con el ejecutable solo es necesario abrirlo y tener los archivos de respaldo en la misma carpeta en caso de tenerlos.

Conclusiones

Al tratarse de la primera versión de un sistema de punto de venta, se han implementado las funciones más esenciales para mantener un control básico pero funcional del inventario, el registro de productos y las ventas. Durante el desarrollo del proyecto se identificaron

oportunidades de mejora a nivel funcional y en la aplicación de buenas prácticas. Por ejemplo, el uso de diccionarios fue una manera más eficiente para manipular y acceder eficientemente a los datos.

Este proyecto ha sido de mucha utilidad para aplicar conceptos de la materia de Conceptos Avanzados de Computación, como la programación concurrente con hilos y el manejo de estructuras de datos. Aunque el sistema actual es una versión inicial, ofrece una base estable y funcional sobre la cual se pueden construir futuras mejoras. En resumen, el desarrollo de este software no sólo permitió cumplir con los requisitos del proyecto, sino que también fortaleció significativamente mis conocimientos y mi capacidad para aplicarlos en un caso real.

Tratándose de la primera versión de un sistema de punto de venta se cuenta con las funciones más esenciales para mantener un buen registro de ventas, de productos y de inventario. A través del proceso de desarrollo se pensó en qué maneras el sistema podría mejorarse tanto en cuanto a funciones como en buenas prácticas de desarrollo, como por ejemplo se encontró en casos que el uso de diccionarios facilita el uso de ciertas operaciones que manipulan datos así como funciones que podrían haber sido útiles para los hilos como la generación de un reporte de ventas diario. En futuras versiones el sistema podrá seguir expandiéndose pero en este momento las funciones entregadas se entregan estables aportando un entendimiento más avanzado de cómo podemos usar hilos en casos reales para que estos sean útiles y no solo un nice to have así como la manipulación de estructuras de datos como los diccionarios y como su uso puede facilitar el manejo de datos. Finalmente puedo decir que desarrollar este software me reforzó los conocimientos para la materia de conceptos avanzados de computación.